TabMixNN: A Unified Deep Learning Framework for Structural Mixed Effects Modeling on Tabular Data

Deniz Akdemir
deniz.akdemir.work@gmail.com

October 16, 2026

Abstract

In this article we describe TabMixNN, a PyTorch implementation of neural prediction models for grouped tabular data. A formula-based encoder combines a fixed-effect design with variational random effects, followed by a neural backbone and outcome-specific heads. The implementation provides eight selectable random-effect covariance structures and ten outcome families, including discrete and continuous survival outcomes. We describe its noncentered variational objective, the dimensions of its encoder, and the sequential static–dynamic structural block used by the default GSEM implementation. Evaluation distinguishes held-out observations from known groups from prediction for entirely unseen groups. Reproducible experiments on five regression and four classification datasets compare TabMixNN with classical mixed models, a fixed-effects neural control, XGBoost and Random Forest. Benefits depend on the prediction setting: random effects improve known-group regression relative to the fixed neural control, whereas performance for unseen groups is mixed. A scalar linear Sleepstudy check gives predictions close to an independently fitted ML mixed model. This agreement does not establish REML equivalence. Single-feature ablations provide descriptive sensitivity summaries, and latent variational draws alone are not full posterior predictive uncertainty.

1 Introduction

Tabular data with hierarchical or grouped structure appears across diverse scientific domains, from longitudinal clinical trials to spatial ecology and genomic studies. Classical statistical approaches, particularly linear mixed-effects models (LMMs) [Bates et al., 2014, Pinheiro and Bates, 2006], have long been the gold standard for such data, offering interpretable fixed and random effects while accounting for within-group correlation. However, these methods often assume linear relationships and parametric distributions, limiting their applicability to complex, high-dimensional data.

Deep learning has revolutionized many domains through its ability to learn complex nonlinear patterns, yet its application to tabular data with mixed-effects structure remains limited. Existing deep learning frameworks for tabular data [Gorishniy et al., 2021, Arik and Pfister, 2021] typically do not account for hierarchical structure or random effects, while classical mixed-effects implementations [Bates et al., 2014] lack the representational power of neural networks.

We introduce TabMixNN (Mixed Effects Neural Networks for Tabular Data), a unified framework that bridges this gap by integrating:

- **Mixed-effects modeling**: Variational random effects with flexible covariance structures including autoregressive (AR), kinship, and Gaussian process kernels
- **Modern neural architectures**: Generalized Structural Equation Models (GSEM) with static/dynamic structure learning and spatial-temporal manifold networks
- **Diverse outcome families**: Support for ten outcome families covering regression, classification, multivariate, multi-label and survival outcomes
- **Interpretability**: Single-feature ablation values, variance decomposition, and parameter extraction for model understanding
- **Accessible interface**: R-style formula syntax familiar to statisticians

The framework is designed to be modular, extensible, and accessible to both statisticians familiar with classical mixed-effects models and machine learning practitioners seeking to incorporate hierarchical structure into deep learning workflows.

1.1 Contributions

Our main contributions include:

1. A modular architecture that cleanly separates mixed-effects encoding, backbone networks, and outcome-specific heads, enabling flexible composition of components
2. Variational treatment of random effects with support for eight selectable covariance structures (IID, AR1, ARMA, Compound Symmetry, Unstructured slopes, Kronecker, Kinship, Gaussian Process)
3. Two complementary backbone architectures: GSEM with ordered triangular structure and gated attention, and Manifold networks for spatial-temporal data
4. Support for ten implemented outcome families enabling multitask learning across heterogeneous outcome types
5. Interpretability tools bridging deep learning and classical statistics (parameter extraction, variance components, single-feature ablation values)
6. An open-source PyTorch implementation with extensive documentation

1.2 Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work in mixed-effects modeling and deep learning for tabular data. Section 3 presents the TabMixNN framework in detail, including the mixed-effects encoder, backbone architectures, outcome families, and training procedures. Section 4 describes the software interface and typical usage patterns. Section 5 presents empirical benchmarks on diverse datasets. Section 6 concludes with discussion and future directions.

2 Background and Related Work

2.1 Classical Mixed-Effects Models

Linear mixed-effects models (LMMs) extend linear regression to handle grouped or hierarchical data by partitioning variation into fixed effects (population-level parameters) and random effects (group-specific deviations). The canonical LMM formulation is:

$$\mathbf{y}_i = \mathbf{X}_i\boldsymbol{\beta} + \mathbf{Z}_i\mathbf{u}_i + \boldsymbol{\epsilon}_i \tag{1}$$

where $\mathbf{y}_i$ is the response vector for group i, $\mathbf{X}_i$ contains fixed-effects covariates, $\mathbf{Z}_i$ contains random-effects covariates, $\boldsymbol{\beta}$ are fixed-effects coefficients, $\mathbf{u}_i \sim \mathcal{N}(0, \boldsymbol{\Sigma}_u)$ are random effects, and $\boldsymbol{\epsilon}_i \sim \mathcal{N}(0, \sigma^2\mathbf{I})$ is residual error.

Estimation typically proceeds via maximum likelihood (ML) or restricted maximum likelihood (REML) using Expectation-Maximization (EM) or direct optimization. Popular implementations include R's `lme4` [Bates et al., 2014] and `nlme` [Pinheiro and Bates, 2006] packages.

Extensions include:

- **Generalized LMMs (GLMMs)**: Allow non-Gaussian outcomes via link functions [McCulloch and Searle, 2001]
- **Nonlinear mixed-effects models (NLMMs)**: Nonlinear mean functions [Lindstrom and Bates, 1990]
- **Flexible covariance structures**: AR, ARMA, compound symmetry for longitudinal data
- **Spatial models**: Gaussian processes, SPDE for spatial correlation [Lindgren et al., 2011]

Despite their interpretability and statistical rigor, these methods struggle with high-dimensional data and complex nonlinear relationships.

2.2 Deep Learning for Tabular Data

Recent deep learning methods for tabular data (e.g., TabNet [Arik and Pfister, 2021], FT-Transformer [Gorishniy et al., 2021], SAINT [Somepalli et al., 2021], TabPFN [Hollmann et al., 2023]) have shown promise on standard tabular benchmarks. However, these methods do not explicitly model random effects or hierarchical structure, limiting their applicability to grouped/longitudinal data. For such structured data, gradient boosting methods like XGBoost remain the predominant baseline in practice.

2.3 Structural Equation Models and Causal Learning

Structural Equation Models (SEMs) [Bollen, 1989] express relationships among variables via systems of equations:

$$\boldsymbol{\eta} = (\mathbf{I} - \mathbf{B})^{-1}\boldsymbol{\xi} \tag{2}$$

where $\boldsymbol{\eta}$ are endogenous variables, $\boldsymbol{\xi}$ are exogenous variables, and $\mathbf{B}$ is an adjacency matrix encoding directed relationships. Acyclicity is enforced by requiring $\mathbf{B}$ to be strictly lower-triangular.

Recent work has integrated SEMs with neural networks:

- **NOTEARS** [Zheng et al., 2018]: Continuous optimization for DAG structure learning
- **DAG-GNN** [Yu et al., 2019]: Graph neural networks with DAG constraints
- **DeepSEM** [Biza et al., 2023]: Neural networks with structural constraints

TabMixNN extends these ideas by combining structural learning with mixed-effects and flexible outcome modeling.

2.4 Spatial-Temporal Deep Learning

For spatial and temporal data, specialized architectures include:

- **Convolutional Neural Networks (CNNs)**: Exploit spatial locality
- **Recurrent Neural Networks (RNNs/LSTMs)**: Model temporal dependencies
- **Graph Neural Networks (GNNs)**: Learn on irregular spatial graphs
- **Neural Processes** [Garnelo et al., 2018]: Meta-learning for function approximation

Exact dense Gaussian process (GP) inference has cubic computational cost in the number of observations. SPDE approximations [Lindgren et al., 2011] support spatial modeling through sparse precision matrices. TabMixNN includes a grid-based precision construction, although its current graph-precision path uses dense Cholesky factorization.

3 Methods

TabMixNN implements a modular three-stage architecture:

1. **Mixed-Effects Encoder**: Transforms raw tabular data into neural representations incorporating fixed and random effects
2. **Backbone Architecture**: Processes representations through neural networks (GSEM or Manifold backbones)
3. **Output Heads**: Map backbone features to outcome-specific parameters via family-specific transformations

3.1 Mixed-Effects Encoder

The encoder transforms tabular data into neural representations by combining fixed effects (population-level features) and random effects (group-specific embeddings).

3.1.1 Fixed Effects

Let $\mathbf{X} \in \mathbb{R}^{n \times p_X}$ be the fixed-effects design compiled from the formula. Numeric transformations and categorical dummy levels are learned from the training data. Here p_X is the design width, including the requested intercept and expanded categorical columns, rather than the number of original variables. The main estimator uses a single projection,

$$\mathbf{H}_{\text{fixed}} = \mathbf{X}\mathbf{W}_f, \qquad \mathbf{W}_f \in \mathbb{R}^{p_X \times d}, \quad \mathbf{H}_{\text{fixed}} \in \mathbb{R}^{n \times d}. \tag{3}$$

The formula intercept is included exactly once; the encoder projection has no additional bias. Separate categorical embedding tables are available in a legacy input-layer class, but are not used by this formula-based estimator.

3.1.2 Random Effects

Index random-effect terms by t. Term t has G_t training levels and S_t intercept/slope columns, with d latent channels per coefficient. The implementation uses noncentered coordinates:

$$p(\mathbf{v}_t) = \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad q(\mathbf{v}_t) = \mathcal{N}(\mathbf{m}_t, \operatorname{diag}(\exp(2\mathbf{a}_t))), \qquad \mathbf{u}_t = \mathbf{L}_t(\boldsymbol{\psi}_t)\mathbf{v}_t. \tag{4}$$

The vectors are reshaped to $G_t \times S_t \times d$ arrays. The factor $\mathbf{L}_t$ acts over levels and/or slopes, separately across latent channels, and defines the effect prior $\boldsymbol{\Sigma}_t = \mathbf{L}_t\mathbf{L}_t^T$. Thus a diagonal posterior in innovation coordinates need not be diagonal in effect coordinates. Training draws use $\mathbf{v}_t = \mathbf{m}_t + \exp(\mathbf{a}_t) \odot \boldsymbol{\epsilon}_t$ with standard-normal $\boldsymbol{\epsilon}_t$.

There is one variational row per observed training level. Index zero is reserved for prediction lookup and contributes no latent variable or KL term. For an unrelated unseen group the default prediction uses zero effects. The optional unknown-embedding setting does not supply subject-specific evidence for a group with no observed outcomes. Related levels supplied with a kinship or GP specification instead use the conditional-mean construction below.

3.1.3 Constraint Enforcement and Identifiability

The optional `enforce_centering` setting subtracts the training-level mean of transformed effects in both training and prediction. If $\mathbf{C}$ denotes this projection, the effect covariance becomes $\mathbf{C}\boldsymbol{\Sigma}_t\mathbf{C}^T$ while the KL remains on the original innovations. This defines a projected latent-variable model; it is not the ordinary unconstrained LMM, and unweighted centering need not make random effects orthogonal to every fixed-effect column in an unbalanced design. Centering is disabled by default.

With a scalar linear identity decoder, a Gaussian outcome, fixed covariance parameters, no centering or extra penalties, and unrestricted variational means, the optimum mean satisfies the Gaussian mixed-model normal equations. The mean-field restriction still affects posterior covariance and can affect jointly estimated variance parameters. TabMixNN does not implement REML. Nonlinear decoders and finite stochastic optimization do not inherit exact BLUP or variance-component recovery guarantees from the linear special case.

3.1.4 Covariance Structures

Eight covariance types are selectable through `cov_struct_map`. All use the innovation prior above; covariance parameters are point estimates learned through the transformed likelihood.

IID Covariance Independent levels and slopes use $u_{gsj} = \sigma_s v_{gsj}$ with $\sigma_s = \exp(\ell_s) > 0$.

AR1 Covariance For each slope and latent channel, $\rho_s = \tanh(r_s)$ and $\sigma_s = \exp(\ell_s)$ define

$$u_1 = \frac{\sigma_s v_1}{\sqrt{1-\rho_s^2}}, \qquad u_g = \rho_s u_{g-1} + \sigma_s v_g. \tag{5}$$

Apart from numerical stabilization, the marginal covariance is $\mathrm{Cov}(u_g, u_h) = \sigma_s^2 \rho_s^{|g-h|}/(1-\rho_s^2)$. Here σ_s is an innovation standard deviation. The ordering is over configured grouping levels, not automatically over repeated observation times within each group; it must be scientifically meaningful.

ARMA Covariance The finite-start recursion is

$$u_g = \sum_{r=1}^{p} \phi_r u_{g-r} + \sigma_s \left(v_g + \sum_{r=1}^{q} \theta_r v_{g-r} \right). \tag{6}$$

Pre-sample states and innovations are zero. Bounded partial correlations are converted to stable AR coefficients by a recursion; MA coefficients are bounded directly, without an invertibility constraint. Consequently the finite-start covariance is not the stationary covariance of the AR1 implementation.

Compound Symmetry For $G_t > 1$, each slope uses

$$\mathbf{\Sigma}_s = \sigma_s^2[(1-\rho_s)\mathbf{I} + \rho_s \mathbf{1}\mathbf{1}^T], \qquad -\frac{1}{G_t - 1} < \rho_s < 1. \tag{7}$$

A transformed parameter enforces this interval, and a jittered Cholesky factor maps innovations to effects.

Unstructured Slope Covariance (UN) At each independent grouping level, the intercept and slope vector has covariance $\mathbf{\Sigma}_{\text{slope}} = \mathbf{L}_s \mathbf{L}_s^T$. The lower-triangular factor has unrestricted off-diagonal entries and diagonal entries $\text{softplus}(r_{ss}) + 10^{-3}$. The same factor is applied to every level and latent channel. This permits correlated random intercepts and slopes.

Kronecker Product For a supplied group relationship matrix and a slope covariance factor,

$$\mathbf{\Sigma}_t = (\mathbf{K}_{\text{group}} + \delta \mathbf{I}) \otimes (\mathbf{L}_s \mathbf{L}_s^T). \tag{8}$$

The implementation applies the group and slope factors successively. This is a separable level–slope covariance, not an unrestricted covariance for every pair of coefficients.

Kinship Covariance A supplied positive-semidefinite relationship matrix gives $\mathbf{u}_{\text{train},s} = \sigma_s \,\text{chol}(\mathbf{K}_{TT} + \delta\mathbf{I})\mathbf{v}_s$. For supplied related but unobserved levels, the prediction is

$$\widehat{\mathbf{u}}_{\text{new},s} = \mathbf{K}_{\text{new},T}(\mathbf{K}_{TT} + \delta\mathbf{I})^{-1}\widehat{\mathbf{u}}_{T,s}. \tag{9}$$

This is a plug-in conditional mean under the fitted prior, not a full predictive distribution: conditional residual covariance for the new levels is not added. With variational fitting and a nonlinear decoder it should not be called an exact classical BLUP.

Gaussian Process Covariance The GP option uses a supplied coordinate array and an RBF kernel,

$$K_s(\mathbf{x}_g, \mathbf{x}_h) = \exp\left(-\frac{\|\mathbf{x}_g - \mathbf{x}_h\|^2}{2\ell_s^2}\right), \qquad \ell_s > 0, \tag{10}$$

with learned slope-specific amplitude σ_s and lengthscale. The training covariance is $\sigma_s^2(\mathbf{K}_{s,TT} + \delta\mathbf{I})$; Cholesky transforms and related-level predictions follow the same construction as the kinship option. Numerical jitter is retained in the implemented solves.

3.1.5 Combined Representation

Let $g_t(i)$ be observation i's level for term t, and let $\mathbf{Z}_t \in \mathbb{R}^{n\times S_t}$ contain its numeric slope values (including a column of ones for a random intercept). The encoder is

$$H_{\text{encoder},ij} = \sum_{k=1}^{p_X} X_{ik}W_{f,kj} + \sum_t \sum_{s=1}^{S_t} Z_{t,is}U_{t,g_t(i),s,j}, \qquad j = 1,\ldots,d. \tag{11}$$

Gathering group rows produces an $n \times S_t \times d$ tensor; multiplying by the broadcast slope design and summing over s gives an $n \times d$ contribution. The fixed and random terms therefore have the same width. Any external scaling of slope covariates must be fitted on training data and applied consistently to all partitions.

3.2 Backbone Architectures

TabMixNN offers two complementary backbone architectures tailored to different data types and modeling goals.

3.2.1 GSEM Backbone

The GSEM backbone operates on latent channels. Its ordered structural transformations describe predictive dependencies; without additional identification assumptions they do not identify causal relationships among measured variables.

Static Structure For an observation in layer ℓ, let $\boldsymbol{\xi}^{(\ell)}$ denote the projected hidden vector (after layer normalization when enabled). The static transformation is

$$\mathbf{u}^{(\ell)} = (\mathbf{I} - \mathbf{B}_s^{(\ell)})^{-1}\boldsymbol{\xi}^{(\ell)}, \qquad \mathbf{B}_s = \text{tril}(\mathbf{B}_{\text{raw}}, -1). \tag{12}$$

The ordering of channels is fixed and the matrix is acyclic by construction. The NOTEARS expression $\text{tr}\{\exp(\mathbf{B}_s \odot \mathbf{B}_s)\} - d$ is identically zero for this parameterization; the corresponding compatibility method returns zero. Sparsity regularization can penalize $\|\mathbf{B}_s\|_1$, but no nontrivial acyclicity optimization is required here.

Dynamic Structure A multi-head attention map computes query, key and value projections from its input $\mathbf{u}$. For each head the attention scores have the form

$$\mathbf{A}(\mathbf{u}) = \text{softmax}\left(\frac{\mathbf{Q}(\mathbf{u})\mathbf{K}(\mathbf{u})^T}{\sqrt{d_k}}\right). \tag{13}$$

Value and output projections are part of the resulting map Attention($\mathbf{u}$). In the GSEM hidden-layer configuration, feature mixing is enabled for flat tabular representations; score matrices alone are not the complete dynamic linear operator.

Combined Static–Dynamic Structure The default `CompositeStructuralModel` is a sequential gated block:

$$\mathbf{u}^{(\ell)} = (\mathbf{I} - \mathbf{B}_s^{(\ell)})^{-1}\boldsymbol{\xi}^{(\ell)}, \tag{14}$$

$$\mathbf{a}^{(\ell)} = \text{Attention}^{(\ell)}(\mathbf{u}^{(\ell)}), \tag{15}$$

$$\mathbf{h}^{(\ell)} = \alpha_\ell \mathbf{u}^{(\ell)} + (1 - \alpha_\ell)\mathbf{a}^{(\ell)}, \qquad 0 < \alpha_\ell < 1. \tag{16}$$

The attention input is the already resolved $\mathbf{u}^{(\ell)}$, so the forward pass is not circular. Disabling the static component sets $\mathbf{u} = \boldsymbol{\xi}$; disabling the dynamic component sets $\mathbf{a} = \mathbf{u}$. A separate `HybridStructuralModel` implements iterative coupled updates. It is a different model, not an equivalent formulation of this gated block. A spectral penalty alone does not establish convergence when attention depends on the iterated state.

Layer Components The hidden layer applies a linear projection, optional normalization and structural block, GELU activation, optional dropout, and a residual addition when input and output dimensions agree. The benchmark's main model disables structural components; the Sleepstudy component comparison evaluates them explicitly.

3.2.2 Manifold Backbone

For spatial and temporal data with regular grid structure (e.g., images, time series, spatial fields), the Manifold backbone implements spatial-temporal modeling via Stochastic Partial Differential Equations (SPDEs).

Architecture The Manifold backbone consists of L blocks, each operating on an N-dimensional grid:

$$\mathbf{H}^{(\ell)} = \text{ManifoldBlock}^{(\ell)}(\mathbf{H}^{(\ell-1)}, \text{grid}_\ell) \tag{17}$$

Each block includes:

1. Grid Mapping

Map input to grid positions:

$$\mathbf{G}^{(\ell)} = \text{Reshape}(\mathbf{W}_{\text{grid}}^{(\ell)}\mathbf{H}^{(\ell-1)}, \text{grid_shape}_\ell) \tag{18}$$

2. SPDE Kernel (Optional)

Apply spatial smoothness via Matérn Gaussian process approximation. The Matérn field $u(\mathbf{s})$ satisfies:

$$(\kappa^2 - \Delta)^{\alpha/2} u(\mathbf{s}) = \mathcal{W}(\mathbf{s}) \tag{19}$$

where $\mathcal{W}$ is Gaussian white noise, κ controls the spatial range, and α controls smoothness (Matérn parameter $\nu = \alpha - d/2$ for dimension d).

Discretizing on a grid via finite differences yields a sparse precision matrix:

$$\mathbf{Q} = \tau(\kappa^2\mathbf{I} - \boldsymbol{\Delta})^\alpha + \delta\mathbf{I} \tag{20}$$

where $\boldsymbol{\Delta}$ is the discrete Laplacian, $\tau > 0$ scales precision and $\delta > 0$ is numerical jitter. The separable Kronecker path uses a different precision approximation. Random effects are sampled:

$$\mathbf{u}_{\text{spatial}} = \mathbf{Q}^{-1/2}\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}) \tag{21}$$

using dense Cholesky decomposition in the implemented graph-precision path: $\mathbf{Q} = \mathbf{L}\mathbf{L}^T$, so $\mathbf{u} = \mathbf{L}^{-T}\boldsymbol{\epsilon}$.

3. SEM Layer (Optional)

Position-aware structural dependencies on the grid:

$$\boldsymbol{\eta}(\mathbf{s}) = (\mathbf{I} - \mathbf{B}(\mathbf{s}))^{-1}\boldsymbol{\xi}(\mathbf{s}) \tag{22}$$

where $\mathbf{B}(\mathbf{s})$ incorporates spatial priors via RBF kernels based on grid distances.

4. Aggregation

Blocks can be aggregated via:

- Concatenation: $[\mathbf{H}^{(1)}, \ldots, \mathbf{H}^{(L)}]$
- Summation: $\sum_\ell \mathbf{H}^{(\ell)}$
- Attention: $\sum_\ell \alpha_\ell \mathbf{H}^{(\ell)}$ where α_ℓ are learned weights

3.3 Outcome Families and Output Heads

TabMixNN supports 10 outcome families, each with a dedicated output head that transforms backbone features to family-specific parameters.

3.3.1 General Formulation

For outcome k, the output head computes:

$$\boldsymbol{\theta}_k = g_k(\mathbf{W}_k\mathbf{H}_{\text{backbone}} + \mathbf{b}_k) \tag{23}$$

where g_k is a family-specific link function and $\boldsymbol{\theta}_k$ are distribution parameters.

The loss for outcome k is the negative log-likelihood:

$$\mathcal{L}_k = -\log p(\mathbf{y}_k|\boldsymbol{\theta}_k) \tag{24}$$

3.3.2 Implemented Families

1. Gaussian Continuous outcomes with identity link:

$$\mu = \mathbf{W}\mathbf{H} + \mathbf{b} \tag{25}$$

$$y \sim \mathcal{N}(\mu, \sigma^2) \tag{26}$$

2. Binomial Binary classification with logit link:

$$\text{logit}(p) = \mathbf{WH} + \mathbf{b} \tag{27}$$
$$y \sim \text{Bernoulli}(p) \tag{28}$$

3. Multinomial Multi-class classification with softmax:

$$\mathbf{p} = \text{softmax}(\mathbf{WH} + \mathbf{b}) \tag{29}$$
$$y \sim \text{Categorical}(\mathbf{p}) \tag{30}$$

4. Poisson Count data with log link:

$$\log(\lambda) = \mathbf{WH} + \mathbf{b} \tag{31}$$
$$y \sim \text{Poisson}(\lambda) \tag{32}$$

5. Negative Binomial Overdispersed count data:

$$\log(\mu) = \mathbf{WH} + \mathbf{b} \tag{33}$$
$$y \sim \text{NegBin}(\mu, \phi) \tag{34}$$

where $\phi > 0$ is predicted on the log scale and $\text{Var}(y) = \mu + \phi\mu^2$ (NB2 parameterization).

6. Multivariate Gaussian Multiple correlated continuous outcomes:

$$\boldsymbol{\mu} = \mathbf{WH} + \mathbf{b} \tag{35}$$
$$\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \tag{36}$$

where $\boldsymbol{\Sigma}$ is the outcome covariance matrix.

7. Multi-Label Binomial Multiple independent binary outcomes:

$$\mathbf{p} = \text{sigmoid}(\mathbf{WH} + \mathbf{b}) \tag{37}$$
$$y_j \sim \text{Bernoulli}(p_j) \quad \text{independently} \tag{38}$$

8. Discrete-Time Survival For T prespecified intervals the head outputs logits η_{it} and hazards $h_{it} = \text{sigmoid}(\eta_{it})$. For an observed interval t_i and event indicator d_i, the log likelihood is

$$\sum_{t<t_i} \log(1 - h_{it}) + d_i \log h_{it_i} + (1 - d_i) \log(1 - h_{it_i}). \tag{39}$$

Censoring at t_i therefore includes survival through that interval. Targets contain interval indices and event indicators; interval definitions must be supplied consistently.

9. Discrete-Time Competing Risks For C causes, the head outputs TC logits. A per-interval softmax includes a fixed zero logit for no event and C cause logits. With probabilities $p_{it0}, \ldots, p_{itC}$, an event of cause c_i contributes $\sum_{t<t_i} \log p_{it0} + \log p_{it_ic_i}$; censoring contributes survival through t_i. Causes are mutually exclusive within each interval.

10. Continuous-Time Survival The implemented continuous family is Weibull proportional hazards. It learns a common shape $k > 0$ and scale $\lambda > 0$, with a predicted log-hazard ratio η_i:

$$H_i(t) = (t/\lambda)^k \exp(\eta_i), \qquad h_i(t) = \frac{k}{t} H_i(t), \qquad \log p(t_i, d_i) = d_i \log h_i(t_i) - H_i(t_i). \quad (40)$$

These likelihoods assume censoring can be ignored conditional on modeled covariates. Implementation of the survival families does not establish their predictive performance; the experiments below evaluate Gaussian and binomial outcomes.

3.3.3 Multitask Learning with Output SEM

Several outcomes can share an encoder and backbone. If output SEM is enabled, the first raw head coordinate of each outcome is stacked into a vector $\mathbf{r}_i$ and transformed as

$$\widetilde{\mathbf{r}}_i = (\mathbf{I} - \mathbf{B}_{\text{output}})^{-1} \mathbf{r}_i. \quad (41)$$

The output matrix is strictly lower triangular in outcome order. Auxiliary head coordinates bypass this transformation, after which each family applies its own parameter mapping. The available modes are independent heads, learned triangular structure, and a user-specified lavaan-style structure restricted to the supported ordering. These links couple raw prediction scores; they do not establish causal relations among outcomes.

3.4 Training Procedure

3.4.1 Objective Function

For N training rows, let $\mathcal{O}_i$ be the outcomes observed in row i. The negative evidence lower bound divided by N is

$$\mathcal{J} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}_q \left[\sum_{k \in \mathcal{O}_i} -\log p_k(y_{ik} \mid \mathbf{v}, \mathbf{x}_i) \right] + \frac{1}{N} \mathrm{KL}(q(\mathbf{v}) \| p(\mathbf{v})). \quad (42)$$

Missing outcomes contribute zero likelihood without changing the row denominator. For vector-valued outcomes all components must be observed. This likelihood assumes an ignorable missing-data mechanism; informative missingness is not modeled.

KL Divergence and Structured Priors The full KL is computed in innovation coordinates:

$$\mathcal{L}_{\mathrm{KL}} = \frac{1}{2} \sum_{t,j} \left[\exp(2a_{tj}) + m_{tj}^2 - 1 - 2a_{tj} \right], \qquad \mathbf{u}_t = \mathbf{L}_t(\boldsymbol{\psi}_t) \mathbf{v}_t. \quad (43)$$

Here j runs over every training-level, slope and channel coordinate. Structured covariance enters the likelihood through $\mathbf{L}_t$, rather than by comparing a diagonal effect-space posterior directly with $\mathcal{N}(0, \boldsymbol{\Sigma}_t)$. Covariance and network parameters are optimized point estimates.

For a uniformly sampled minibatch $\mathcal{B}$ of B rows, the implemented stochastic objective is

$$\widehat{\mathcal{J}}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \sum_{k \in \mathcal{O}_i} -\log p_k(y_{ik} \mid \mathbf{v}, \mathbf{x}_i) + \lambda_{\mathrm{KL}} \frac{\mathcal{L}_{\mathrm{KL}}}{N}, \qquad \mathbf{v} \sim q. \quad (44)$$

Algorithm 1 TabMixNN variational training

Require: Training data, formula, covariance specifications, update budget

Fit design transformations on training rows only
Initialize variational, covariance, backbone and head parameters
Initialize Adam parameter groups
for each training update with minibatch $\mathcal{B}$ **do**
    Sample innovations $\mathbf{v}_t = \mathbf{m}_t + \exp(\mathbf{a}_t) \odot \boldsymbol{\epsilon}_t$
    Transform effects $\mathbf{u}_t = \mathbf{L}_t(\boldsymbol{\psi}_t)\mathbf{v}_t$
    Compute encoder, backbone and outcome-head outputs
    Compute mean row NLL plus $\lambda_{\mathrm{KL}}\mathcal{L}_{\mathrm{KL}}/N$
    Add any explicitly requested structural penalties
    Backpropagate, optionally clip gradients, and take an Adam step
end for

The full global KL is included at every update. The default multiplier is one; parameter-count normalization and automatic group-size damping are disabled. The optional rule $\lambda_{\mathrm{KL}} = 1/\bar{n}_g$ is empirical tempering and has no ELBO or exact-BLUP derivation. Explicit outcome weights or alternative likelihood reductions likewise change the displayed objective.

Additional Regularization Structural sparsity and optional contraction penalties are added separately. The static triangular block has zero acyclicity penalty. Adam decay on deterministic weights is also separate from the likelihood/KL objective; variational parameters, covariance parameters, likelihood dispersion parameters and biases are excluded from that decay. The manuscript experiments use zero weight decay and no additional structural penalties.

3.4.2 Optimization

The generic defaults are 200 epochs, batch size 256 and learning rate 0.001. Validation-based early stopping is optional and requires separate validation data; a training budget is not a convergence certificate. The benchmark protocol in Section 5 specifies its own fixed full-batch budget and does not select models using test outcomes.

3.5 Inference and Prediction

3.5.1 Point Predictions

For a known group, prediction uses the transformed variational mean $\widehat{\mathbf{u}}_t = \mathbf{L}_t\mathbf{m}_t$. An unrelated unseen group receives zero latent effects by default; supplied related levels use a conditional mean through the relationship kernel. These are plug-in predictions: for a nonlinear decoder, evaluating at the latent mean is generally different from averaging predictions over its distribution.

A separate support/query interface can adapt new IID or UN groups using labeled support observations while freezing the trained network. It minimizes support negative log likelihood plus one half the squared standard-normal innovation norm, i.e., conditional MAP adaptation. Support and query rows must be disjoint and query outcomes must not

be used. This is a different evaluation regime from the no-support group holdout reported here.

3.5.2 Uncertainty Quantification

Drawing $\mathbf{v}_t$ from the fitted variational distribution, transforming it through $\mathbf{L}_t$, and passing it through the decoder describes latent-induced variation in fitted conditional means. Quantiles of these draws are not full prediction intervals: observation noise, uncertainty in point-estimated network and covariance parameters, and conditional residual variation for related unseen levels are omitted. The experiments report seed variability of prediction metrics and do not claim calibrated posterior predictive coverage.

3.6 Model Interpretation

TabMixNN provides several tools for model interpretation, bridging deep learning with classical statistical summaries.

3.6.1 Parameter Extraction

For models with limited nonlinearity (e.g., shallow networks), we can extract approximate fixed-effects coefficients as average derivatives on the primary predictor scale. A local Taylor expansion illustrates the interpretation:

$$f(\mathbf{x}) \approx f(\bar{\mathbf{x}}) + \nabla f(\bar{\mathbf{x}})^T(\mathbf{x} - \bar{\mathbf{x}}) \tag{45}$$

The implementation averages local derivatives over supplied observations; these are not global GLM coefficients for a nonlinear network. Scalar linear identity-head models instead allow coefficients in their formula units.

3.6.2 Variance Decomposition

For the scalar linear Gaussian special case, conditional on the design matrices,

$$\mathrm{Var}(\mathbf{y} \mid \mathbf{X}, \mathbf{Z}) = \mathbf{Z}\mathbf{G}\mathbf{Z}^T + \sigma_\epsilon^2\mathbf{I}, \tag{46}$$

where $\mathbf{G}$ contains the random-coefficient covariance, including any intercept–slope covariances. Only for a single independent random intercept with a common residual variance does this simplify to the familiar within-group correlation

$$\mathrm{ICC} = \frac{\sigma_b^2}{\sigma_b^2 + \sigma_\epsilon^2}. \tag{47}$$

With random slopes the correlation depends on the observations' covariates. With nonlinear decoders, encoder-space covariance scales are not automatically response-scale variance components or ICCs, and an additive response-variance decomposition does not follow from the encoder alone.

3.6.3 Feature Ablation Values

The `feature_ablation_values` routine replaces one fixed-design column at a time by its mean in a background sample drawn from the supplied frame, then records the prediction difference. Group indices and random-slope design columns remain unchanged. A training frame must therefore be supplied when a training-derived background is intended; the routine does not automatically retrieve one. In the figures we use all training rows as the background.

These are descriptive single-column sensitivity measures, not Shapley values: no feature coalitions are averaged and contributions need not sum to the prediction minus a baseline [Lundberg and Lee, 2017]. In particular, perturbing the fixed Days column while retaining its random-slope input does not estimate a subject's total Days effect or random-slope coefficient. Random-effect ablations instead remove one random term at a time and report group-averaged prediction changes. Such changes may interact and need not be additive. Coalition-based edge attribution is not implemented.

3.6.4 Model Summary

A summary table reports:

- Fixed effects: Linearized coefficient estimates (average derivatives on the primary predictor scale; not conventional global coefficients for nonlinear models)
- Random effects: Encoder-space variance components and correlations (not automatically outcome-unit variance components or ICCs in nonlinear models)
- Model fit: Conditional plug-in log-likelihood only; standard errors, p-values, AIC, and BIC are not implemented

4 Software Interface and Usage

TabMixNN is implemented in PyTorch and provides an scikit-learn compatible API with additional R-style formula interface.

4.1 Installation

```
pip install tabmixnn
# or for development:
git clone https://github.com/dakdemir-nmdp/tabmixnn
cd tabmixnn
pip install -e .
```

4.2 Basic Usage

4.2.1 Simple Regression

```
from tabmixnn import TabMixNNRegressor

# Formula-based specification
```

```
model = TabMixNNRegressor(
    formula="y ~ x1 + x2 + (1|group_id)",
    hidden_dims=[64, 32],
    epochs=100,
    lr=0.001
)

# Fit
model.fit(train_data)

# Predict
predictions = model.predict(test_data)
```

4.2.2 Binary Classification

```
from tabmixnn import TabMixNNClassifier

model = TabMixNNClassifier(
    formula="y ~ x1 + x2 + (x1|patient_id)",
    hidden_dims=[128, 64, 32],
    epochs=50
)
model.fit(train_data)
probabilities = model.predict_proba(test_data)
```

4.3 Advanced Features

4.3.1 Multitask Learning

```
from tabmixnn import TabMixNN, OutcomeSpec, Gaussian, Binomial

outcomes = [
    OutcomeSpec(
        name="continuous",
        y="outcome1",
        family=Gaussian()
    ),
    OutcomeSpec(
        name="binary",
        y="outcome2",
        family=Binomial()
    )
]

model = TabMixNN(
    formula="~ x1 + x2 + (1|subject_id)",
    outcomes=outcomes,
    output_sem_mode='learned',  # Learn inter-outcome dependencies
    hidden_dims=[64, 32]
```

```
)

model.fit(train_data)
predictions = model.predict(test_data, as_dict=True)
# Returns: {'continuous': ..., 'binary': ...}
```

4.3.2 Custom Covariance Structures

For correlated intercepts and slopes within independent subjects:

```
from tabmixnn import TabMixNN, Gaussian
model = TabMixNN(
    formula="y ~ time + (1 + time | subject)",
    family=Gaussian(),
    cov_struct_map={'subject': {'type': 'UN'}}
)
```

AR1 and ARMA order grouping levels, so arbitrary subject identifiers should not be treated as a temporal sequence. A GP specification requires `coords` aligned with `levels`; a kinship specification requires `matrix_tensor` (or `K_group`) and its corresponding `levels`. Related unseen levels must be included in that supplied ordering.

4.3.3 Structural Learning

```
# Fit an ordered latent dependency structure
model = TabMixNN(
    formula="y ~ x1 + x2 + x3",
    family=Gaussian(),
    use_static_structure=True,
    use_dynamic_structure=False,
    lambda_sparse=0.01,   # Sparsity penalty
    hidden_dims=[64, 64]
)

model.fit(train_data)

# Extract learned adjacency matrix
B_matrix, normalized_magnitudes = model.extract_sam()
# Normalized magnitudes are not posterior edge probabilities.
```

4.3.4 Spatial-Temporal Modeling

```
# Manifold backbone with SPDE
model = TabMixNN(
    formula="~ temperature + (1|station_id)",
    outcomes=[OutcomeSpec(
        name="y",
        y="precipitation",
        family=Gaussian()
    )],
```

```
    architecture='manifold',
    manifold_configs=[
        {
            'grid_shape': (20, 20),  # 2D spatial grid
            'use_spde': True,
            'spde_alpha': 2,         # Smoothness
            'spde_kappa_init': 0.3,  # Range
            'use_sem': False
        }
    ]
)
```

4.4 Model Interpretation

```
# Summary statistics (lme4-style)
summary_df = model.summary(data=train_data)
print(summary_df)

# Parameter extraction
params = model.extract_parameters(
    data=train_data,
    linearize=True
)
print("Fixed effects:", params['fixed_effects'])
print("Variance components:", params['variance_components'])

# Encoder-space covariance summaries
print(model.get_variance_summary())

# Fixed-design sensitivity on a training-derived background
ablation = model.feature_ablation_values(
    train_data, background_size=len(train_data)
)
print(ablation['ablation_values'])
print(model.random_effect_ablations(train_data))
```

4.5 Model Persistence

```
# Save
model.save("model.pt")

# Load
from tabmixnn import TabMixNN
loaded_model = TabMixNN.load("model.pt")

# A new fit starts from scratch; it is not warm-start training.
loaded_model.fit(new_data, epochs=50)
```

5 Benchmarks

5.1 Benchmark Datasets

Nine public datasets were evaluated: five regression and four binary classification tasks (Table 1). The dataset inventory and all reported tables refer to the same experiments. Dataset sources, checksums, model predictors and split row positions are recorded with the results.

Table 1: Datasets and held-out samples. p counts substantive predictors before one-hot encoding, excluding group identity and the intercept; G counts groups. R: regression; C: classification. A and B give held-out rows; parentheses in B give held-out groups.

Dataset	n	p	G	Task	A	B (G)
Sleepstudy	180	1	18	R	36	40 (4)
ChickWeight	578	2	50	R	143	144 (12)
Machines	54	1	6	R	12	9 (1)
Oxboys	234	1	26	R	52	54 (6)
Loblolly	84	1	14	R	14	18 (3)
Aids2	2,843	3	4	C	710	588 (1)
Males	4,360	4	545	C	1090	1088 (136)
VerbAgg	7,584	5	316	C	1896	1896 (79)
Train	2,929	8	235	C	642	723 (58)

Regression Sleepstudy predicts reaction time in milliseconds from Days, with subject-specific intercepts and slopes. ChickWeight predicts weight in grams from Time and categorical Diet, with chick intercepts. Machines predicts productivity score from categorical Machine, with worker intercepts. Oxboys predicts height in centimetres from standardized age, with subject intercepts and slopes; Occasion is not supplied to any method. Loblolly predicts tree height in feet from age, with seed-source intercepts.

Classification Aids2 predicts recorded final vital status from age, sex and transmission category, grouped by state. Observation times differ, so this is a descriptive binary task, not a fixed-horizon survival-risk analysis. Males predicts union membership from schooling, experience, ethnicity and industry, grouped by individual. VerbAgg predicts the dichotomized response from Anger, Gender, behavior type, situation and mode, grouped by respondent. Train predicts choice1 from the two alternatives' price, time, changes and comfort, grouped by respondent; the choice identifier is excluded. These outcomes and model predictors have no missing values in the downloaded versions.

5.2 Experimental Setup

5.2.1 Two Prediction Regimes

Regime A: within-group observation holdout. A random 25% of observations, rounded down with a minimum of one, is held out separately within each group having at least two observations. At least one observation remains for every tested group. Singleton

groups remain training-only. This evaluates prediction for known groups using their fitted information; it does not evaluate prospective forecasting beyond the observed time range.

Regime B: out-of-group holdout. A random 25% of complete groups, rounded down with a minimum of one, is held out. No labeled support from these groups is available. TabMixNN uses zero random effects, and the classical mixed model uses its fixed part. For nonlinear models these are plug-in predictions at zero latent effects, not marginal integration over new-group variation. Kinship/GP extrapolation is not evaluated in this suite.

Each regime uses one fixed split with seed 20261015, shared by all methods. Test outcomes are used only for final evaluation. All categorical coding and numeric/target standardization are fitted on the training partition. Unseen categorical levels receive the training reference coding. The recorded split manifests permit exact reconstruction.

5.2.2 Models and Training Budgets

TabMixNN uses one hidden layer of width 32, embedding dimension 8, UN covariance and a Gaussian or binomial head. Each fit uses 2,000 full-batch Adam updates with learning rate 0.01, KL multiplier one, zero weight decay, no dropout, no centering and no extra structural penalties. Full batches retain source row order. The structural backbone options are disabled in the main comparison. Three initialization seeds (42, 123 and 456) are evaluated, without selecting the best seed. There is no test-driven hyperparameter search or early stopping.

The **fixed neural control** uses the same hidden architecture and budget with random terms removed. The **classical mixed baseline** uses the same fixed and random designs: Gaussian LMMs are fitted by ML using `lme4::lmer`; binomial GLMMs use `glmer` with the Laplace approximation [Bates et al., 2014]. Known-group predictions include conditional fitted effects, whereas Regime B uses zero effects. Final optimizer convergence checks passed. The Aids2 random-intercept fit was singular in both regimes, a reported boundary estimate rather than a discarded run.

XGBoost uses 200 trees, maximum depth 6 and learning rate 0.05 [Chen and Guestrin, 2016]. **Random Forest** uses 200 trees and minimum leaf size two. Both use seed 42. All methods receive the same substantive predictors; in Regime A, trees additionally receive training-fitted one-hot group indicators to make known-group information available. Trees receive no group indicators in Regime B. These are fixed comparator configurations, not exhaustively tuned methods.

5.2.3 Metrics and Reproducibility

Regression RMSE and MAE are reported in original outcome units after reversing training-target standardization; R^2 is also retained in the result files. Classification tables report AUROC and Brier score, with accuracy and log loss retained in those files. Lower Brier scores indicate smaller squared probability error; they alone do not establish calibration. Raw RMSE is not averaged across datasets with different units.

Neural table entries are the mean and sample SD of metrics across three initialization seeds on the same test split. This SD does not estimate sampling uncertainty and is not a confidence interval. Classical and tree baselines use a single deterministic fit. No significance tests across datasets are claimed.

The executed environment used PyTorch 2.5.1, scikit-learn 1.9.1, XGBoost 3.2.0, R 4.4.1 and lme4 1.1-35.5. Scripts, versions, source checksums, per-seed metrics, baseline

diagnostics and split manifests are provided in `experiments/`. The canonical suite is `run_manuscript_benchmarks.py`; tables are generated directly from its aggregate outputs. Source records and per-row predictions are cached locally and are not distributed with the manuscript.

5.3 Results

5.3.1 Within-Group Regression

Table 2: Regression RMSE in original outcome units (lower is better). Regime A: observation holdout within known groups. Neural entries are mean ± sample SD over three initialization seeds on one fixed split; other methods use one deterministic fit. These SDs are not confidence intervals.

Dataset (unit)	TabMixNN	Fixed NN	Mixed	XGBoost	RF
Sleepstudy (ms)	33.597 ± 2.217	48.007 ± 0.269	32.190	31.184	34.618
ChickWeight (g)	5.580 ± 0.103	40.278 ± 0.389	30.948	17.260	31.733
Machines (score)	1.021 ± 0.122	6.428 ± 0.000	3.431	0.656	2.528
Oxboys (cm)	0.847 ± 0.009	8.274 ± 0.004	0.926	1.864	4.430
Loblolly (ft)	0.812 ± 0.147	1.909 ± 0.090	2.582	1.989	1.882

TabMixNN has lower RMSE than the fixed neural control on all five known-group regression tasks. The comparison with other methods is task-dependent: XGBoost gives lower RMSE on Sleepstudy and Machines, whereas TabMixNN gives lower RMSE on ChickWeight, Oxboys and Loblolly. The large ChickWeight difference from the linear mixed model is consistent with the benefit of a nonlinear fitted response on this split; it does not by itself isolate a biological growth mechanism.

5.3.2 Out-of-Group Regression

Table 3: Regression RMSE in original outcome units (lower is better). Regime B: entirely unseen groups, without labeled support. Neural entries are mean ± sample SD over three initialization seeds on one fixed split; other methods use one deterministic fit. These SDs are not confidence intervals.

Dataset (unit)	TabMixNN	Fixed NN	Mixed	XGBoost	RF
Sleepstudy (ms)	29.985 ± 1.696	29.550 ± 0.090	29.446	29.644	29.407
ChickWeight (g)	41.224 ± 0.598	42.758 ± 0.024	41.078	42.751	42.702
Machines (score)	3.889 ± 0.140	5.072 ± 0.001	5.072	5.073	5.143
Oxboys (cm)	11.098 ± 0.903	11.307 ± 0.000	11.313	11.511	11.449
Loblolly (ft)	1.473 ± 0.165	1.229 ± 0.067	2.650	1.164	1.165

Without observed outcomes from the test groups, the advantage is smaller. TabMixNN gives lower mean RMSE than XGBoost on three of five datasets (ChickWeight, Machines and Oxboys), but not on Sleepstudy or Loblolly. Machines has only one held-out worker. On Oxboys, TabMixNN's mean R^2 is approximately -0.58, and the other comparators also have negative R^2; a small RMSE advantage here should not be interpreted as useful

generalization. The contrast between Tables 2 and 3 shows why the two prediction settings must be reported separately.

5.3.3 Within-Group Classification

Table 4: Classification AUROC (higher is better) and Brier score (lower is better). Regime A: observation holdout within known groups. Neural entries are mean ± sample SD over three initialization seeds on one fixed split; other methods use one deterministic fit. These SDs are not confidence intervals.

Dataset	Metric	TabMixNN	Fixed NN	Mixed	XGBoost	RF
Aids2	AUC	0.548 ± 0.011	0.573 ± 0.010	0.572	0.553	0.543
Aids2	Brier	0.237 ± 0.003	0.233 ± 0.004	0.231	0.236	0.241
Males	AUC	0.893 ± 0.008	0.710 ± 0.003	0.910	0.808	0.899
Males	Brier	0.103 ± 0.003	0.170 ± 0.001	0.095	0.138	0.109
VerbAgg	AUC	0.838 ± 0.001	0.696 ± 0.004	0.807	0.772	0.809
VerbAgg	Brier	0.165 ± 0.002	0.224 ± 0.003	0.179	0.197	0.183
Train	AUC	0.832 ± 0.022	0.739 ± 0.025	0.786	0.775	0.753
Train	Brier	0.179 ± 0.014	0.227 ± 0.014	0.192	0.193	0.211

Known-group classification also shows heterogeneous results. TabMixNN improves AUROC over the fixed neural control for Males, VerbAgg and Train, but not Aids2. The classical GLMM has the highest AUROC on Males, while TabMixNN has the highest mean AUROC on VerbAgg and Train. Probability-error rankings can differ from AUROC rankings, as the Brier scores show.

5.3.4 Out-of-Group Classification

Table 5: Classification AUROC (higher is better) and Brier score (lower is better). Regime B: entirely unseen groups, without labeled support. Neural entries are mean ± sample SD over three initialization seeds on one fixed split; other methods use one deterministic fit. These SDs are not confidence intervals.

Dataset	Metric	TabMixNN	Fixed NN	Mixed	XGBoost	RF
Aids2	AUC	0.540 ± 0.008	0.529 ± 0.010	0.489	0.538	0.546
Aids2	Brier	0.241 ± 0.001	0.242 ± 0.002	0.239	0.239	0.239
Males	AUC	0.620 ± 0.020	0.650 ± 0.027	0.651	0.723	0.674
Males	Brier	0.230 ± 0.006	0.211 ± 0.008	0.228	0.183	0.196
VerbAgg	AUC	0.697 ± 0.007	0.679 ± 0.003	0.700	0.668	0.663
VerbAgg	Brier	0.242 ± 0.004	0.230 ± 0.001	0.223	0.238	0.244
Train	AUC	0.737 ± 0.003	0.683 ± 0.010	0.723	0.702	0.668
Train	Brier	0.256 ± 0.003	0.267 ± 0.004	0.215	0.230	0.233

The largest decrease in TabMixNN AUROC occurs for Males when individual-specific fitted information is unavailable. XGBoost performs better on that task. For VerbAgg, the mixed-model and TabMixNN AUROCs are close, while the fixed neural control has lower Brier error. Aids2 has only four grouping levels and holds out one state; all methods have AUROC near 0.5. These small-group results have limited support for broad extrapolation.

5.3.5 Linear Mixed-Model Check

Table 6: Sleepstudy linear-model check: raw RMSE (ms). Linear VI uses a scalar identity decoder and unstructured intercept/slope covariance; lme4 fits the same linear mean and random-effects structure by ML. Near agreement of predictions does not imply identical covariance estimation or REML equivalence.

Regime	Linear VI	LMM (ML)
A	32.503 ± 0.104	32.190
B	29.292 ± 0.141	29.446

On Sleepstudy we additionally fit a scalar linear TabMixNN decoder using the same intercept/slope structure and split as the LMM. Regime-A RMSE is about 32.5 ms for linear VI and 32.2 ms for ML. This is a numerical check of predictions under a simple mean structure. It does not establish equality of variance estimates, posterior uncertainty or REML fitting, and it should not be extrapolated to the nonlinear benchmark model.

5.3.6 Ablation Studies

Table 7: Sleepstudy component comparisons on the same Regime-A split. RMSE is in milliseconds. Entries are mean ± sample SD over three seeds. Except for the scalar linear decoder, all variants use one hidden layer of width 32 and embedding dimension 8; all use the same 2,000-update budget. Structural variants use feature mixing without sparsity or contraction penalties. This is a component comparison, not a hyperparameter search.

Configuration	RMSE ↓	R^2 ↑
Intercept + slope	33.597 ± 2.217	0.597 ± 0.054
Intercept only	33.900 ± 1.646	0.590 ± 0.039
Fixed only	48.007 ± 0.269	0.180 ± 0.009
Linear decoder	32.503 ± 0.104	0.624 ± 0.002
Static	32.312 ± 2.473	0.627 ± 0.058
Dynamic	31.940 ± 0.444	0.637 ± 0.010
Gated static + dynamic	32.013 ± 0.965	0.635 ± 0.022

Table 7 compares seven configurations on the same Sleepstudy observation holdout. Removing random effects increases error relative to the intercept/slope model. Adding static or dynamic latent structure does not show a uniform improvement across seeds, and the comparison does not identify an optimal architecture. The three-seed variability and fixed update budget should be considered when interpreting small differences. No general recommendation about layer count or learning rate is inferred from this single dataset.

5.4 Interpretability Analysis

Figures 1–3 are descriptive plots from a separate fit to all 180 Sleepstudy observations (seed 42, 2,000 updates, the same nonlinear architecture). All training rows form the background. Values are transformed back to milliseconds. The fixed-design Days column is perturbed while random-slope values and group indices remain fixed, as described in Section 3.6.

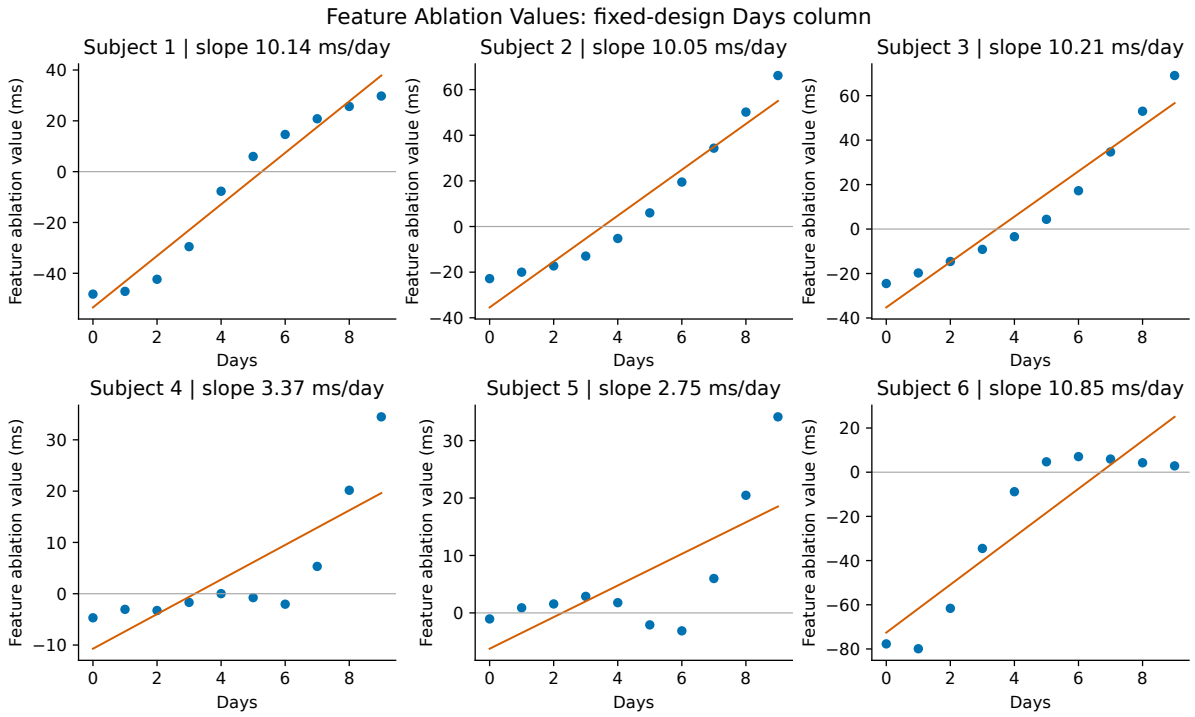


Figure 1: Fixed-design Days ablation values for six subjects, in milliseconds. Each point is the prediction minus its value after replacing the fixed-design Days column by the training-background mean, with the random-slope inputs and group indices held fixed. Lines are descriptive least-squares summaries, with slopes in ms/day. This is a full-data illustration, not held-out prediction assessment or recovery of subject-specific random coefficients. Subjects have anonymous source-order labels.

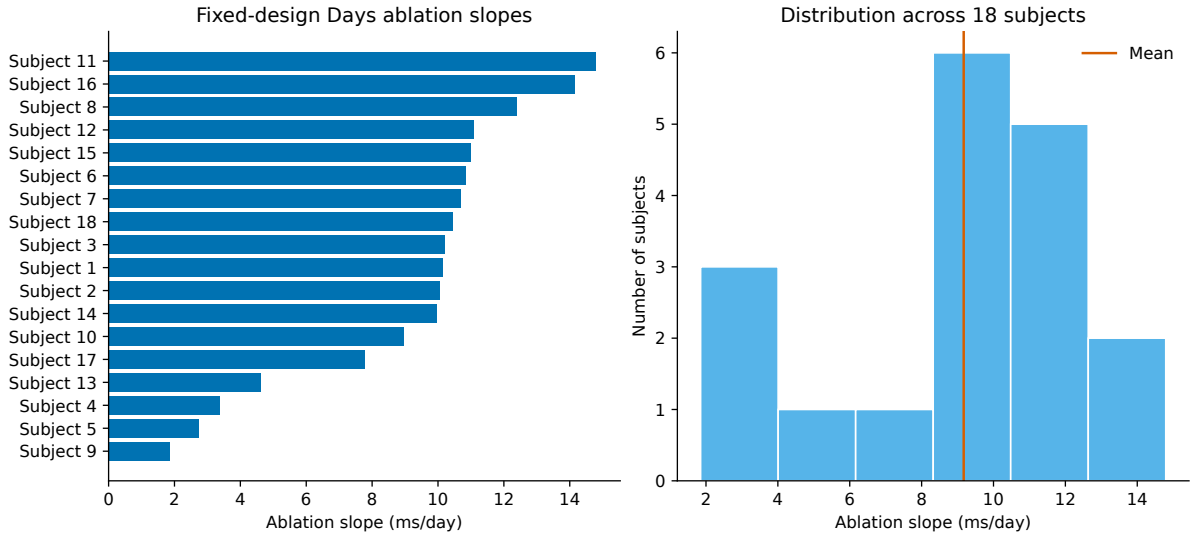


Figure 2: Ablation slopes from the fixed-design Days perturbation across all 18 subjects. Left: ordered descriptive line slopes in ms/day. Right: their distribution, with a line marking the mean. These summaries describe the fitted function under a specified perturbation; they do not estimate the random-slope variance component.

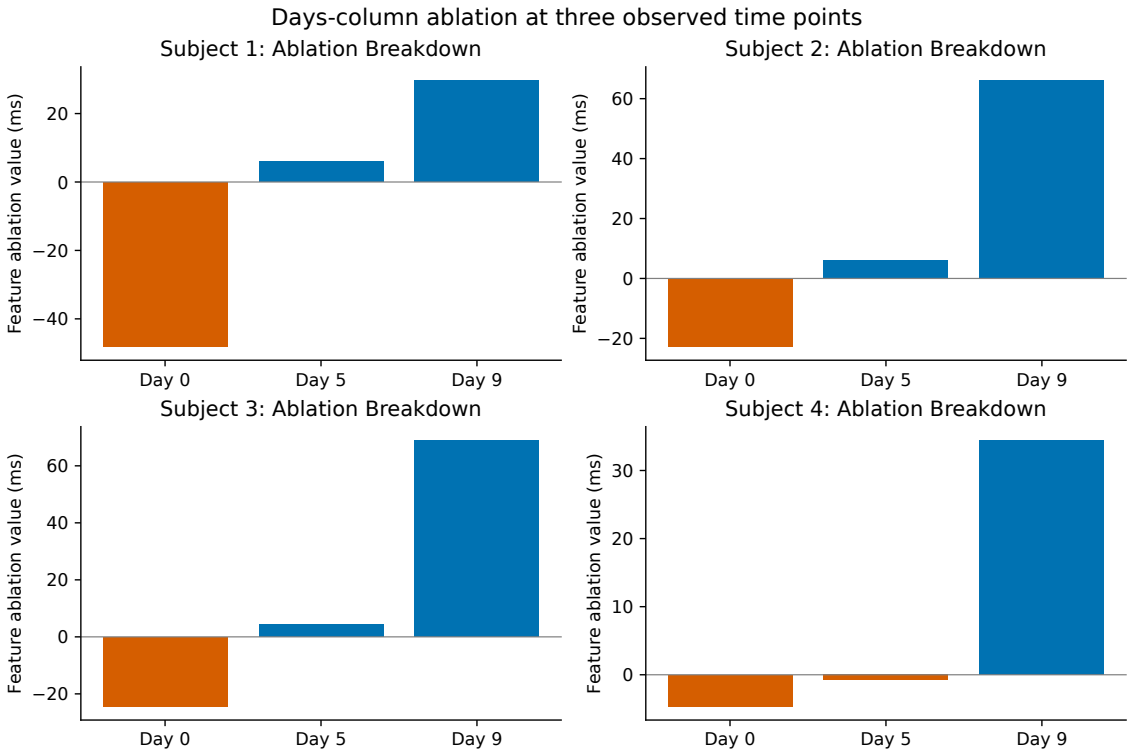


Figure 3: Ablation breakdown for four subjects at observed Days 0, 5 and 9. Bars show the single fixed-design Days-column prediction difference in milliseconds, using the same background and frozen random-slope inputs as Figure 1. Bars describe separate observations and are not accumulated to reconstruct a prediction.

The fitted lines in Figure 1 summarize the plotted perturbations; they are neither random-slope coefficients nor causal effects of sleep deprivation. Figure 2 summarizes their distribution. Figure 3 displays the same column perturbation at three observed days; its bars are not an additive decomposition of a prediction. The complete generator is `create_feature_ablation_plots.py` in the experiments directory.

5.5 Scope of the Empirical Evidence

These experiments evaluate Gaussian regression, binomial classification and one component comparison. They do not evaluate survival prediction, multitask benefit, genomic prediction or the spatial manifold backbone. Fixed splits and limited model configurations also leave sensitivity to split choice and tuning unresolved. The package supports these additional interfaces, but no performance claim for them follows from the present results.

6 Discussion and Conclusion

We have presented TabMixNN, a comprehensive framework for mixed-effects deep learning on tabular data. The framework successfully bridges classical statistical modeling and modern neural networks through:

- **Modular architecture**: Clean separation of encoder, backbone, and output heads enables flexible composition
- **Rich covariance structures**: Support for temporal (AR/ARMA), spatial (GP/SPDE), and genetic (kinship) correlation
- **Diverse outcome families**: Unified interface for regression, classification, and multitask learning
- **Structural regularization**: Ordered triangular blocks and optional sparsity penalties
- **Interpretability**: Parameter extraction, variance decomposition, and single-feature ablation values
- **Accessibility**: R-style formula interface and scikit-learn compatibility

6.1 Limitations and Future Work

Current limitations and directions for future research include:

- **Scalability**: For very large datasets ($n > 10^6$), additional optimizations (distributed training, approximations) may be needed
- **Automatic structure search**: The present triangular parameterization fixes a channel ordering; it does not search all DAG orderings
- **Uncertainty quantification**: Observation-noise integration, uncertainty in network/covariance parameters and predictive coverage require additional work

- **Non-Euclidean data**: Extensions to graphs, manifolds, and other non-tabular structures
- **Identifiability**: Centering does not identify all latent coordinates, and predictive structural edges do not have a causal interpretation without additional assumptions
- **Evaluation scope**: One split per regime and three neural seeds do not measure sampling uncertainty. Small group counts, limited baseline tuning and the absence of external validation limit generalization; survival, multitask and spatial/genomic applications are not benchmarked here
- **Fairness and privacy**: Incorporation of fairness constraints and differential privacy

6.2 Conclusion

TabMixNN combines variational mixed-effects encoding with neural prediction for grouped tabular data. In the nine-dataset evaluation, fitted group effects improve prediction over the fixed neural control on several known-group tasks, while performance for entirely unseen groups varies by dataset. A scalar linear check gives predictions close to an ML mixed model without establishing equivalence of covariance estimates or uncertainty. The implementation supports broader outcome and structural models, but their predictive value requires separate evaluation. Single-feature ablations describe fitted-model sensitivity; they do not identify causal effects.

References

Sercan Ö Arik and Tomas Pfister. Tabnet: Attentive interpretable tabular learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8):6679–6687, 2021.

Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1):1–48, 2014.

Konstantinos Biza, Ioannis Tsamardinos, and Sofia Triantafillou. Deep structural causal models for tractable counterfactual inference. *Advances in Neural Information Processing Systems*, 36, 2023.

Kenneth A Bollen. *Structural equations with latent variables*. John Wiley & Sons, 1989.

Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

Marta Garnelo, Jonathan Schwarz, Dan Rosenbaum, Fabio Viola, Danilo J Rezende, SM Eslami, and Yee Whye Teh. Neural processes. *arXiv preprint arXiv:1807.01622*, 2018.

Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems*, 34:18932–18943, 2021.

Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. Tabpfn: A transformer that solves small tabular classification problems in a second. *arXiv preprint arXiv:2207.01848*, 2023.

Finn Lindgren, Håvard Rue, and Johan Lindström. An explicit link between gaussian fields and gaussian markov random fields: the stochastic partial differential equation approach. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 73(4):423–498, 2011.

Mary J Lindstrom and Douglas M Bates. Nonlinear mixed effects models for repeated measures data. *Biometrics*, pages 673–687, 1990.

Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

Charles E McCulloch and Shayle R Searle. *Generalized, linear, and mixed models*. John Wiley & Sons, 2001.

José Pinheiro and Douglas Bates. *Mixed-effects models in S and S-PLUS*. Springer Science & Business Media, 2006.

Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C Bayan Bruss, and Tom Goldstein. Saint: Improved neural networks for tabular data via row attention and contrastive pre-training. *arXiv preprint arXiv:2106.01342*, 2021.

Yue Yu, Jie Chen, Tian Gao, and Mo Yu. Dag-gnn: Dag structure learning with graph neural networks. *International Conference on Machine Learning*, pages 7154–7163, 2019.

Xun Zheng, Bryon Aragam, Pradeep K Ravikumar, and Eric P Xing. Dags with no tears: Continuous optimization for structure learning. *Advances in Neural Information Processing Systems*, 31, 2018.